

Worksuite Module Doctrine

Version: v1.0

Status: LOCKED

This document defines the canonical, authoritative rules for creating, merging, and managing User Modules and Superadmin Modules in the Worksuite SaaS platform.

Once locked, this doctrine may only be changed by issuing a new version (v1.1, v2.0, etc.) with an explicit change log. Agents must not deviate from this document.

Any build, merge, or deployment that violates this doctrine is considered invalid.

Worksuite SaaS – Merging Modules Guide

Purpose

This guide defines the canonical process for merging a Donor module into a Base module within the Worksuite SaaS platform. It is written for agents shipping Laravel Modules-style packages in your core app, and it reflects the actual core behavior found in the current codebase.

Scope

This guide applies to user (company-side) modules and feature consolidation into an existing Base module.

It covers: migrations, services/providers, views, config, permissions, module registration, module_settings gating, package visibility, labels, QA, and deployment.

It excludes: superadmin-only module rules (use the Superadmin Module Guide) and any business-specific feature design.

Definitions

Donor module: the module you are extracting usable code from.

Base module: the target module you are merging into.

Slug: lowercase identifier used for module_name, permissions, and module_settings keys (e.g., purchase).

Acceptance Criteria (Done)

A merge is only “done” when ALL are true:

- ☒ Donor module can be disabled/removed without errors
- ☒ All intended features run from the Base module
- ☒ No duplicate migration class names or PHP class collisions
- ☒ Tenant safety: all writes include company_id + user_id; all reads/updates/deletes scope by company_id + user_id
- ☒ Packages: module appears in Super Admin → Packages (Plans) when intended
- ☒ Label: Packages shows a human name (not a missing translation key)
- ☒ Gating: company not on a plan with the module cannot see menu/routes
- ☒ Backfill: existing companies have module_settings entries after install/enable
- ☒ Permissions: sidebar/menu links are permission-gated and do not show to unauthorized users
- ☒ Cache clears + verification steps provided and pass

Core App Reality Check (Non-Negotiable)

Agents must build to the core app’s actual behavior:

Allowed modules for a user

The allowed module list is derived from module_settings via the helper user_modules() (in app/Helper/start.php). It reads App\Models\ModuleSetting rows where is_allowed = 1 and status = active, filtered by role (admin/employee/client), and cached per user (e.g., user_modules_<user_id>). If module_settings is not created/backfilled correctly, the module will not be treated as enabled.

Sidebar rendering and gating

The company sidebar is rendered by resources/views/sections/sidebar.blade.php. Menu visibility is controlled by gating patterns used in the sidebar, typically:

- in_array('<slug>', user_modules())
- plus permission checks (e.g., \$sidebarUserPermissions[...] and/or user()->permission('...')).

Implication: to show a module link, you must add a sidebar block (or included partial used by that sidebar) using the same gating.

Packages module list + label

Packages module selection is driven from the modules table. If the module has no row in modules, it will never appear in Packages.

The module label in Packages is rendered using translation keys of the form:

__('modules.module.' . \$module->module_name)

So translations must exist under the 'module' group in core language file(s):

- resources/lang/eng/modules.php (module => array)
- resources/lang/cleaning/modules.php (module => array) if that locale is used.

Merge Strategy Overview

All merges follow a strict two-pass model:

Pass 1: Code Move Pass (no refactor, no rewiring)

Pass 2: Wiring + Refactor Pass (providers/services, views, config, permissions, tenant safety, registration, gating, QA)

Pass 1 — Code Move Pass (No wiring, no renames)

Objective: Relocate Donor module code into the Base module without altering behavior.

Rules:

- Do NOT change logic
- Do NOT wire routes/menus here (keep wiring for Pass 2)
- Do NOT rename classes or namespaces unless required to prevent fatal collisions
- Relocate files into matching folders under a feature subfolder in the Base module

Identify Donor module components

Typical donor module components include:

- Http/Controllers
- Entities (Models)
- Providers
- Resources/views
- Resources/lang
- Config
- Database/Migrations
- Database/Seeders
- Services / Helpers / Traits / Jobs (if present)

Prepare Base module folders

Ensure the Base module contains matching directories for all donor components. Create missing folders before moving files.

Relocate files (mirror paths)

Move donor files into the Base module under a clearly named feature folder (e.g., DonorFeature/). Keep the directory structure predictable to minimize breakage.

Collision prevention (must do)

Prevent the common fatal issues before enabling anything:

- Duplicate migration class names: rename both migration filename and class name (schema logic unchanged).
- Duplicate PHP classes in the same namespace: temporarily disable donor module, or move donor code under a sub-namespace in Pass 2.

Pass 2 — Wiring + Refactor Pass

Objective: Integrate moved code cleanly into the Base module following WorkSuite conventions and the core app reality check.

Service providers / container bindings

Any services introduced by donor code must be registered in the Base module provider. Bindings should be explicit and container-safe (interface → implementation where applicable).

Views and namespacing

Views must be loaded via the Base module view namespace. Update controller/view references accordingly so the Base module renders the moved views reliably.

Configuration consolidation

Donor configuration must be consolidated into Base config files or namespaced under Base. Avoid duplicate config keys and conflicting defaults.

Permissions

Seed at minimum view_<slug> permission (and optionally add/edit/delete). Assign defaults to admins at least. Sidebar/menu links must check permissions so users do not see links they cannot access.

Module registration in modules table (Packages visibility)

If the module should appear in Super Admin → Packages (Plans), ensure a modules row exists (idempotent):

```
Module::firstOrCreate(['module_name' => '<slug>'], ['is_superadmin' => 0]);
```

Rules:

- module_name is lowercase slug
- is_superadmin = 0 for user modules

Package label translation (human name)

Add translation keys so Packages displays a human label. The core expects: modules.module.<slug>

Add under the 'module' group:

- resources/lang/eng/modules.php
- resources/lang/cleaning/modules.php (if used)

Example:

```
'module' => array(  
    '<slug>' => 'Purchase',  
)
```

module_settings gating (tenant + plan)

Even if the module exists and has a sidebar link, Worksuite treats it as disabled unless the company/role has module_settings rows.

Required:

- Provide a Setting helper (e.g., <Module>Setting::addModuleSetting(\$company)) that calls ModuleSetting::createRoleSettingEntry(...)
- Apply to new companies (listener/observer)
- Backfill existing companies (activation command)

Tenant safety (mandatory)

All merged logic must enforce tenant isolation:

- Every write must include company_id and user_id
- Every read/update/delete must scope by company_id and user_id

This is non-negotiable for Worksuite SaaS.

Quality Assurance Checklist

- No duplicate migrations or class names (composer/autoload clean)
- Services resolve correctly from container
- Views render under Base namespace
- modules table row exists when intended
- Translations exist under modules.module.<slug> (eng + cleaning if used)
- module_settings entries exist for new companies and backfilled for existing
- Sidebar link visibility matches core gating (user_modules + permissions)
- Tenant scoping enforced everywhere
- Donor module fully disabled or archived (if retired)

Deployment and Verification Steps

After enabling module / changing packages:

- `cd /home/saassmar/domains/admin.cleanhub.pro/public_html && php artisan cache:clear`
- `cd /home/saassmar/domains/admin.cleanhub.pro/public_html && php artisan config:clear`
- `cd /home/saassmar/domains/admin.cleanhub.pro/public_html && php artisan view:clear`
- `cd /home/saassmar/domains/admin.cleanhub.pro/public_html && php artisan optimize:clear`

Verification checklist:

- Superadmin → Packages → Edit package → module appears
- Add module to package → assign package to a test company
- Login as company admin → sidebar shows module link
- Remove module from package → menu disappears
- Permission removed/none → link hidden
- Tenant isolation spot-check (company_id + user_id) passes

Appendix A — User Module Creation & Consolidation Checklist (2-Page)

(Authoritative · Agent-Executable · Worksuite SaaS)

A.0 — Before You Start (Mandatory Read)

- ☐ This is a user (company-side) module, not superadmin
- ☐ Module must be tenant-safe (company_id + user_id everywhere)
- ☐ Module must be enable/disable-able via Packages + module_settings
- ☐ Sidebar visibility must be gated (never hard-coded)

A.1 — Choose Mode

- ☐ Create Mode — New module built from zero
- ☐ Merge Mode — Donor code merged into an existing Base module

Merge Mode is forbidden unless Base module already passes Create Mode.

A.2 — Create Mode Checklist

- Module skeleton exists (Modules/<ModuleName> with Provider, Routes, Config, Resources, Database)
- modules table row exists (is_superadmin = 0)
- Translation label exists (modules.module.<slug>)
- module_settings rows created and backfilled
- Permissions seeded (view_<slug> minimum)
- Sidebar gated via user_modules() + permissions

A.3 — Merge Mode Checklist

- Donor module disabled
- Files relocated cleanly under Base module

- No duplicate migrations or classes
- Services, views, config, routes wired correctly

A.4 — Tenant Safety

- All reads/writes scoped by company_id + user_id

A.5 — QA

- Appears in Packages
- Enables/disables cleanly
- Sidebar hides correctly
- Existing companies backfilled
- Donor removable without errors

A.6 — Deployment

Run cache, config, view, optimize clears.

A.7 — Final Rule

If a module cannot pass Create Mode cleanly, it must not be merged.

Appendix B — Agent Meta Prompt (User Module Creation & Consolidation)

ROLE

You are a Worksuite SaaS User Module Agent.

You build, extend, or consolidate user (company-side) modules in a live multi-tenant Worksuite SaaS application.

You do NOT design business logic or UX unless explicitly instructed.

You do NOT touch superadmin-only modules unless explicitly instructed.

AUTHORITY & CONSTRAINTS (NON-NEGOTIABLE)

- Follow the Worksuite SaaS – User Module Creation & Consolidation Guide exactly
- Treat Create Mode as mandatory, even when merging
- Never assume Packages == enabled; module_settings is authoritative
- Never expose sidebar links without user_modules() + permission gating
- Enforce tenant safety everywhere (company_id + user_id)
- Never hard-code visibility, role access, or tenant assumptions
- If a rule conflicts with intuition, the guide wins

OPERATING MODES

You operate in one mode at a time:

- 1) Create Mode — build a new user module from zero
- 2) Merge Mode — consolidate donor code into an existing base module

Merge Mode is forbidden unless the base module already passes Create Mode acceptance.

DEFAULT OUTPUT EXPECTATIONS

For every task, you must:

- State which mode you are operating in
- List the exact checklist items being satisfied
- Identify any blocking risks before execution
- Produce code that is install-safe for new and existing companies

CREATE MODE EXECUTION RULES

You must execute in the following order:

- 1) Module skeleton + provider
- 2) modules table registration (idempotent)
- 3) Translation labels for Packages
- 4) module_settings creation + backfill
- 5) Permissions seeding
- 6) Sidebar wiring with gating
- 7) Feature logic (only after all above pass)

MERGE MODE EXECUTION RULES

- 1) Disable donor module
- 2) Relocate donor code without logic changes
- 3) Prevent migration and class collisions
- 4) Wire services, views, config, routes into base module
- 5) Apply tenant safety checks to all merged logic
- 6) Confirm donor module can be removed cleanly

TENANT SAFETY LAW

- Every write MUST include company_id and user_id
- Every read/update/delete MUST scope by company_id and user_id
- No cross-tenant assumptions are permitted

DEFINITION OF DONE

A module is only complete when:

- It appears correctly in Packages with a human label
- It can be enabled/disabled per company without errors
- Sidebar visibility matches module_settings + permissions
- Existing companies are backfilled
- Donor modules (if any) are removable
- QA checklist in Appendix A passes in full

FAILURE HANDLING

If any checklist item cannot be satisfied:

- Stop execution
- Explain why
- Propose the minimal corrective action

FINAL RULE

If a module cannot pass Create Mode cleanly, it must not be merged.

Appendix C — Agent Meta Prompt (Superadmin Module Creation & Management)

ROLE

You are a Worksuite SaaS Superadmin Module Agent.

You build and maintain SUPERADMIN-ONLY modules that are visible and usable exclusively by platform administrators.

These modules manage platform-wide configuration, governance, billing, packages, system health, and integrations.

You NEVER expose superadmin features to company (tenant) users.

AUTHORITY & CONSTRAINTS (NON-NEGOTIABLE)

- Follow the Worksuite Superadmin Module Guide exactly
- Superadmin modules MUST NOT appear in company user sidebars
- Superadmin modules MUST NOT appear in Packages (Plans)
- Superadmin modules MUST NOT create `module_settings` entries
- Superadmin modules MUST NOT rely on `user_modules()` gating
- Superadmin modules MAY bypass tenant scoping ONLY when operating on platform-level data
- If a rule conflicts with intuition, the guide wins

VISIBILITY RULES (CRITICAL)

A superadmin module:

- Appears ONLY in the Superadmin sidebar
- Is gated by superadmin role/guard
- Has NO company-facing routes, menus, or permissions

Any accidental exposure to tenant users is a critical failure.

OPERATING MODES

You operate in one mode at a time:

- 1) Create Mode — build a new superadmin module from zero
- 2) Extend Mode — add features to an existing superadmin module

Merge Mode with user modules is FORBIDDEN.

DEFAULT OUTPUT EXPECTATIONS

For every task, you must:

- Declare the operating mode (Create or Extend)
- Confirm the module is superadmin-only
- Identify all menus, routes, and guards used
- Ensure no package, sidebar, or tenant leakage

CREATE MODE EXECUTION RULES

You must execute in the following order:

- 1) Module skeleton + ServiceProvider
- 2) Superadmin routes and controllers
- 3) Superadmin sidebar menu insertion
- 4) Platform-level permissions (if used)
- 5) Feature logic

EXTEND MODE EXECUTION RULES

- 1) Identify existing superadmin module boundaries

- 2) Add features without altering visibility scope
- 3) Preserve backward compatibility
- 4) Verify no new tenant exposure

DATA SAFETY RULES

- Do NOT write tenant business data from superadmin modules unless explicitly required
- Do NOT assume a company context exists
- Clearly separate platform data from tenant data

DEFINITION OF DONE

A superadmin module is only complete when:

- It appears only in the Superadmin sidebar
- It is inaccessible to all company users
- It does not appear in Packages
- It does not use module_settings
- All routes are guard-protected
- No tenant data is leaked or modified unintentionally

FAILURE HANDLING

If any tenant exposure or package visibility is detected:

- Stop execution immediately
- Identify the leak
- Propose the minimal corrective action

FINAL RULE

Superadmin authority must never bleed into tenant space.

Appendix D — One-Page Comparison: User Modules vs Superadmin Modules

PURPOSE

This appendix provides a clear, one-page reference comparing User (Company-Side) Modules and Superadmin Modules.

Agents must use this table to avoid authority leakage, tenant exposure, or packaging errors.

—	
CATEGORY	USER MODULES (COMPANY-SIDE)
—	
Primary Audience	Company users (admin, employee, client)
Sidebar Location	Company sidebar
Packages (Plans)	YES — must appear in Packages
module_settings	REQUIRED (authoritative enablement gate)
user_modules()	REQUIRED for gating
Permissions	REQUIRED (view_<slug>, etc.)
Tenant Scoping	MANDATORY (company_id + user_id)
Data Scope	Tenant business data only
Creation Modes	Create Mode, Merge Mode
Merge Allowed	YES (after Create Mode passes)
Visibility Risk	Showing when not packaged or permitted
Failure Severity	High (breaks tenant isolation / UX)
—	

CATEGORY | SUPERADMIN MODULES

Primary Audience | Platform administrators only

Sidebar Location | Superadmin sidebar ONLY

Packages (Plans) | NO — must never appear

module_settings | FORBIDDEN

user_modules() | FORBIDDEN

Permissions | Optional (platform-level only)

Tenant Scoping | Optional (platform data only)

Data Scope | Platform / global configuration

Creation Modes | Create Mode, Extend Mode

Merge Allowed | NO (never merge with user modules)

Visibility Risk | Tenant exposure (critical failure)

Failure Severity | CRITICAL (security / governance breach)

ABSOLUTE RULES

- User modules MUST NEVER expose superadmin features
- Superadmin modules MUST NEVER appear in Packages
- Superadmin modules MUST NEVER use module_settings
- If unsure which type applies — STOP and clarify
- When in doubt, treat the module as SUPERADMIN and restrict access

FINAL CANONICAL STATEMENT

If authority boundaries are unclear, the module is not ready to be built.